



Universidad Peruana de Ciencias Aplicadas

Ingeniería de Software

Periodo: 202610

1ASI0732 | Diseño de Experimentos de Ingeniería de Software

NRC: 16879

Docente: Alex Humberto S  nchez Ponce

Informe del Trabajo Final

Startup: Stoq

Producto: StockWise

Integrantes

U   Luciana Carolina Choquehuanca Nu  ez

U   Ronald Joel Peralta Chipa

U202210973   Sanchez Rios, Camila Cristina

U   Fabiola Del Rocio Salda  a Ayala

U  

Abril, 2026

Registro de Versiones del Informe

Version	Fecha	Autor	Descripcion de modificaciones
---------	-------	-------	-------------------------------

Project Report Collaboration Insights

Enlace del repositorio del informe

Enlace de los repositorios de la organizacion

Contenido

Tabla de contenidos

- [Registro de Versiones del Informe](#)
- [Project Report Collaboration Insights](#)
 - [Enlace del repositorio del informe](#)
 - [Enlace de los repositorios de la organizacion](#)
- [Contenido](#)
 - [Tabla de contenidos](#)
- [Student Outcome](#)
- [Part I: As-Is Software Project](#)
- [Capitulo I: Introduccion](#)
 - [1.1. Startup Profile](#)
 - [1.1.1. Descripcion de la Startup](#)
 - [1.1.2. Perfiles de integrantes del equipo](#)
 - [1.2. Solution Profile](#)
 - [1.2.1. Antecedentes y problematica](#)
 - [1.2.2. Lean UX Process](#)
 - [1.2.2.1. Lean UX Problem Statements](#)
 - [1.2.2.2. Lean UX Assumptions](#)
 - [1.2.2.3. Lean UX Hypothesis Statements](#)
 - [1.2.2.4. Lean UX Canvas](#)
 - [1.3. Segmentos objetivo](#)
- [Capitulo II: Requirements Elicitation & Analysis](#)
 - [2.1. Competidores](#)
 - [2.1.1. Analisis competitivo](#)
 - [2.1.2. Estrategias y tacticas frente a competidores](#)
 - [2.2. Entrevistas](#)
 - [2.2.1. Diseno de entrevistas](#)
 - [2.2.2. Registro de entrevistas](#)
 - [2.2.3. Analisis de entrevistas](#)
 - [2.3. Needfinding](#)
 - [2.3.1. User Personas](#)
 - [2.3.2. User Task Matrix](#)
 - [2.3.3. User Journey Mapping](#)
 - [2.3.4. Empathy Mapping](#)
 - [2.3.5. As-is Scenario Mapping](#)
 - [2.4. Ubiquitous Language](#)
- [Capitulo III: Requirements Specification](#)
 - [3.1. To-Be Scenario Mapping](#)
 - [3.2. User Stories](#)
 - [3.3. Product Backlog](#)
 - [3.4. Impact Mapping](#)
- [Capitulo IV: Product Design](#)

- 4.1. Style Guidelines
 - 4.1.1. General Style Guidelines
 - 4.1.2. Web Style Guidelines
 - 4.1.3. Mobile Style Guidelines
 - 4.1.3.1. iOS Mobile Style Guidelines
 - 4.1.3.2. Android Mobile Style Guidelines
- 4.2. Information Architecture
 - 4.2.1. Organization Systems
 - 4.2.2. Labeling Systems
 - 4.2.3. SEO Tags and Meta Tags
 - 4.2.4. Searching Systems
 - 4.2.5. Navigation Systems
- 4.3. Landing Page UI Design
 - 4.3.1. Landing Page Wireframe
 - 4.3.2. Landing Page Mock-up
- 4.4. Mobile Applications UX/UI Design
 - 4.4.1. Mobile Applications Wireframes
 - 4.4.2. Mobile Applications Wireflow Diagrams
 - 4.4.3. Mobile Applications Mock-ups
 - 4.4.4. Mobile Applications User Flow Diagrams
- 4.5. Mobile Applications Prototyping
 - 4.5.1. Android Mobile Applications Prototyping
 - 4.5.2. iOS Mobile Applications Prototyping
- 4.6. Web Applications UX/UI Design
 - 4.6.1. Web Applications Wireframes
 - 4.6.2. Web Applications Wireflow Diagrams
 - 4.6.3. Web Applications Mock-ups
 - 4.6.4. Web Applications User Flow Diagrams
- 4.7. Web Applications Prototyping
- 4.8. Domain-Driven Software Architecture
 - 4.8.1. Software Architecture Context Diagram
 - 4.8.2. Software Architecture Container Diagrams
 - 4.8.3. Software Architecture Components Diagrams
- 4.9. Software Object-Oriented Design
 - 4.9.1. Class Diagrams
 - 4.9.2. Class Dictionary
- 4.10. Database Design
 - 4.10.1. Relational/Non-Relational Database Diagram
- Capitulo V: Product Implementation
 - 5.1. Software Configuration Management
 - 5.1.1. Software Development Environment Configuration
 - 5.1.2. Source Code Management
 - 5.1.3. Source Code Style Guide & Conventions
 - 5.1.4. Software Deployment Configuration
 - 5.2. Product Implementation & Deployment
 - 5.2.1. Sprint Backlogs

- 5.2.2. Implemented Landing Page Evidence
 - 5.2.3. Implemented Frontend-Web Application Evidence
 - 5.2.4. Acuerdo de Servicio - SaaS
 - 5.2.5. Implemented Native-Mobile Application Evidence
 - 5.2.6. Implemented RESTful API and/or Serverless Backend Evidence
 - 5.2.7. RESTful API documentation
 - 5.2.8. Team Collaboration Insights
- 5.3. Video About-the-Product
- Part II: Verification, Validation & Pipeline
- Capitulo VI: Product Verification & Validation
 - 6.1. Testing Suites & Validation
 - 6.1.1. Core Entities Unit Tests
 - 6.1.2. Core Integration Tests
 - 6.1.3. Core Behavior-Driven Development
 - 6.1.4. Core System Tests
 - 6.2. Static testing & Verification
 - 6.2.1. Static Code Analysis
 - 6.2.1.1. Coding standard & Code conventions
 - 6.2.1.2. Code Quality & Code Security
 - 6.2.2. Reviews
 - 6.3. Validation Interviews
 - 6.3.1. Diseno de Entrevistas
 - 6.3.2. Registro de Entrevistas
 - 6.3.3. Evaluaciones segun heurísticas
 - 6.4. Auditoria de Experiencias de Usuario
 - 6.4.1. Auditoria realizada
 - 6.4.1.1. Informacion del grupo auditado
 - 6.4.1.2. Cronograma de auditoria realizada
 - 6.4.1.3. Contenido de auditoria realizada
 - 6.4.2. Auditoria recibida
 - 6.4.2.1. Informacion del grupo auditor
 - 6.4.2.2. Cronograma de auditoria recibida
 - 6.4.2.3. Contenido de auditoria recibida
 - 6.4.2.4. Resumen de modificaciones para subsanar hallazgos
- Capitulo VII: DevOps Practices
 - 7.1. Continuous Integration
 - 7.1.1. Tools and Practices
 - 7.1.2. Build & Test Suite Pipeline Components
 - 7.2. Continuous Delivery
 - 7.2.1. Tools and Practices
 - 7.2.2. Stages Deployment Pipeline Components
 - 7.3. Continuous deployment
 - 7.3.1. Tools and Practices
 - Recomendaciones
 - Consideraciones adicionales
 - 7.3.2. Production Deployment Pipeline Components

- 7.4. Continuous Monitoring
 - 7.4.1. Tools and Practices
 - 7.4.2. Monitoring Pipeline Components
 - 7.4.3. Alerting Pipeline Components
 - 7.4.4. Notification Pipeline Components
- Part III: Experiment-Driven Lifecycle
- Capitulo VIII: Experiment-Driven Development
 - 8.1. Experiment Planning
 - 8.1.1. As-Is Summary
 - 8.1.2. Raw Material: Assumptions, Knowledge Gaps, Ideas, Claims
 - 8.1.3. Experiment-Ready Questions
 - 8.1.4. Question Backlog
 - 8.1.5. Experiment Cards
 - 8.2. Experiment Design
 - 8.2.1. Hypotheses
 - 8.2.2. Domain Business Metrics
 - 8.2.3. Measures
 - 8.2.4. Conditions
 - 8.2.5. Scale Calculations and Decisions
 - 8.2.6. Methods Selection
 - 8.2.7. Data Analytics: Goals, KPIs and Metrics Selection
 - 8.2.8. Web and Mobile Tracking Plan
 - 8.3. Experimentation
 - 8.3.1. To-Be User Stories
 - 8.3.2. To-Be Product Backlog
 - 8.3.3. Pipeline-supported, Experiment-Driven To-Be Software Platform Lifecycle
 - 8.3.3.1. To-Be Sprint Backlogs
 - 8.3.3.2. Implemented To-Be Landing Page Evidence
 - 8.3.3.3. Implemented To-Be Frontend-Web Application Evidence
 - 8.3.3.4. Implemented To-Be Native-Mobile Application Evidence
 - 8.3.3.5. Implemented To-Be RESTful API and/or Serverless Backend Evidence
 - 8.3.3.6. Team Collaboration Insights
 - 8.3.4. To-Be Validation Interviews
 - 8.3.4.1. Diseno de Entrevistas
 - 8.3.4.2. Registro de Entrevistas
 - 8.4. Experiment Aftermath & Analysis
 - 8.4.1. Analysis and Interpretation of Results
 - 8.4.2. Re-scored and Re-prioritized Question Backlog
 - 8.5. Continuous Learning
 - 8.5.1. Shareback Session Artifacts: Learning Workflow
 - 8.6. To-Be Software Platform Pre-launch
 - 8.6.1. About-the-Product Intro Video
- Conclusiones
- Conclusiones y recomendaciones
- Video App Validation
- Video About-the-Team

- [Bibliografia](#)
- [Anexos](#)

Student Outcome

ABET – EAC - Student Outcome 4

Criterio: La capacidad de reconocer responsabilidades éticas y profesionales en situaciones de ingeniería y hacer juicios informados, que deben considerar el impacto de las soluciones de ingeniería en contextos globales, económicos, ambientales y sociales.

En el siguiente cuadro se describe las acciones realizadas y enunciados de conclusiones por parte del grupo, que permiten sustentar el haber alcanzado el logro del ABET – EAC - Student Outcome 4.

Criterio específico	Acciones realizadas	Conclusiones
Reconoce responsabilidad ética y profesional en situaciones de ingeniería de software	Miranda Ayasta, Rogger Faryd	
	AV1:	
	TB1:	
	AV2:	
	TB2:	
	Vargas Javier, Jose Enrique	
	AV1:	
	TB1:	
	AV2:	
	TB2:	
	Sanchez Rios, Camila	
	AV1:	
	TB1:	
	AV2:	
	TB2:	
	Luciana Carolina Choquehuanca Nuñez	
	AV1:	
	TB1:	

AV2:

TB2:

**Ronald Joel Peralta
Chipa**

AV1:

TB1:

AV2:

TB2:

**Emite juicios informados considerando el impacto de las
soluciones de ingeniería de software en contextos
globales, económicos, ambientales y sociales**

**Miranda Ayasta,
Roger Faryd**

AV1:

TB1:

AV2:

TB2:

**Vargas Javier, Jose
Enrique**

AV1:

TB1:

AV2:

TB2:

Sanchez Rios, Camila

AV1:

TB1:

AV2:

TB2:

**Luciana Carolina
Choquehuanca Nuñez**

AV1:

TB1:

AV2:

TB2:

**Ronald Joel Peralta
Chipa**

AV1:

TB1:

AV2:

TB2:

Part I: As-Is Software Project

Capitulo I: Introduccion

1.1. Startup Profile

1.1.1. Descripcion de la Startup

1.1.2. Perfiles de integrantes del equipo

1.2. Solution Profile

1.2.1. Antecedentes y problematica

1.2.2. Lean UX Process

1.2.2.1. Lean UX Problem Statements

1.2.2.2. Lean UX Assumptions

1.2.2.3. Lean UX Hypothesis Statements

1.2.2.4. Lean UX Canvas

1.3. Segmentos objetivo

Capitulo II: Requirements Elicitation & Analysis

2.1. Competidores

2.1.1. Analisis competitivo

2.1.2. Estrategias y tacticas frente a competidores

2.2. Entrevistas

2.2.1. Diseno de entrevistas

2.2.2. Registro de entrevistas

2.2.3. Analisis de entrevistas

2.3. Needfinding

2.3.1. User Personas

2.3.2. User Task Matrix

2.3.3. User Journey Mapping

2.3.4. Empathy Mapping

2.3.5. As-is Scenario Mapping

2.4. Ubiquitous Language

Capitulo III: Requirements Specification

3.1. To-Be Scenario Mapping

3.2. User Stories

3.3. Product Backlog

3.4. Impact Mapping

Capitulo IV: Product Design

4.1. Style Guidelines

4.1.1. General Style Guidelines

4.1.2. Web Style Guidelines

4.1.3. Mobile Style Guidelines

4.1.3.1. iOS Mobile Style Guidelines

4.1.3.2. Android Mobile Style Guidelines

4.2. Information Architecture

4.2.1. Organization Systems

4.2.2. Labeling Systems

4.2.3. SEO Tags and Meta Tags

4.2.4. Searching Systems

4.2.5. Navigation Systems

4.3. Landing Page UI Design

4.3.1. Landing Page Wireframe

4.3.2. Landing Page Mock-up

4.4. Mobile Applications UX/UI Design

4.4.1. Mobile Applications Wireframes

4.4.2. Mobile Applications Wireflow Diagrams

4.4.3. Mobile Applications Mock-ups

4.4.4. Mobile Applications User Flow Diagrams

4.5. Mobile Applications Prototyping

4.5.1. Android Mobile Applications Prototyping

4.5.2. iOS Mobile Applications Prototyping

4.6. Web Applications UX/UI Design

4.6.1. Web Applications Wireframes

4.6.2. Web Applications Wireflow Diagrams

4.6.3. Web Applications Mock-ups

4.6.4. Web Applications User Flow Diagrams

4.7. Web Applications Prototyping

4.8. Domain-Driven Software Architecture

4.8.1. Software Architecture Context Diagram

4.8.2. Software Architecture Container Diagrams

4.8.3. Software Architecture Components Diagrams

4.9. Software Object-Oriented Design

4.9.1. Class Diagrams

4.9.2. Class Dictionary

4.10. Database Design

4.10.1. Relational/Non-Relational Database Diagram

Capitulo V: Product Implementation

5.1. Software Configuration Management

5.1.1. Software Development Environment Configuration

5.1.2. Source Code Management

5.1.3. Source Code Style Guide & Conventions

5.1.4. Software Deployment Configuration

5.2. Product Implementation & Deployment

5.2.1. Sprint Backlogs

5.2.2. Implemented Landing Page Evidence

5.2.3. Implemented Frontend-Web Application Evidence

5.2.4. Acuerdo de Servicio - SaaS

5.2.5. Implemented Native-Mobile Application Evidence

5.2.6. Implemented RESTful API and/or Serverless Backend Evidence

5.2.7. RESTful API documentation

5.2.8. Team Collaboration Insights

5.3. Video About-the-Product

Part II: Verification, Validation & Pipeline

Capitulo VI: Product Verification & Validation

6.1. Testing Suites & Validation

6.1.1. Core Entities Unit Tests

6.1.2. Core Integration Tests

6.1.3. Core Behavior-Driven Development

6.1.4. Core System Tests

6.2. Static testing & Verification

6.2.1. Static Code Analysis

6.2.1.1. Coding standard & Code conventions

6.2.1.2. Code Quality & Code Security

6.2.2. Reviews

6.3. Validation Interviews

6.3.1. Diseno de Entrevistas

6.3.2. Registro de Entrevistas

6.3.3. Evaluaciones segun heurísticas

6.4. Auditoria de Experiencias de Usuario

6.4.1. Auditoria realizada

6.4.1.1. Informacion del grupo auditado

6.4.1.2. Cronograma de auditoria realizada

6.4.1.3. Contenido de auditoria realizada

6.4.2. Auditoria recibida

6.4.2.1. Informacion del grupo auditor

6.4.2.2. Cronograma de auditoria recibida

6.4.2.3. Contenido de auditoria recibida

6.4.2.4. Resumen de modificaciones para subsanar hallazgos

Capitulo VII: DevOps Practices

7.1. Continuous Integration

7.1.1. Tools and Practices

En StockWise usamos GitHub como repositorio central para el control de versiones y la colaboración del equipo. La integración continua se plantea como una práctica clave para validar los cambios realizados en el proyecto antes de integrarlos a ramas principales, asegurando que el código compile correctamente y que las pruebas automatizadas puedan ejecutarse de forma constante.

Las pruebas automatizadas son una parte importante de la calidad del producto; por eso el proceso de CI está organizado para considerar la ejecución de:

Maven — herramienta principal para gestionar dependencias, compilar el backend y ejecutar las pruebas del proyecto.

JUnit — framework utilizado para ejecutar pruebas automatizadas en Java, incluyendo pruebas unitarias y pruebas de integración.

Cucumber (Gherkin) — herramienta utilizada para casos BDD; los archivos `.feature` describen escenarios de comportamiento del sistema y se transforman en pruebas ejecutables.

JUnit Platform — componente utilizado para descubrir y ejecutar las pruebas dentro del entorno Java.

GitHub — plataforma utilizada como repositorio central para almacenar el código fuente, revisar cambios y evidenciar la colaboración del equipo.

GitHub Actions — herramienta considerada para automatizar el proceso de Continuous Integration mediante la ejecución del build y las pruebas en eventos como `push` o `pull_request`.

GitFlow — flujo de trabajo usado para organizar las ramas del proyecto, separando el desarrollo principal, funcionalidades, pruebas y versiones estables.

Conventional Commits — convención usada para mantener mensajes de commit claros, trazables y fáciles de revisar.

Prácticas aplicadas:

- Ejecutar la suite de pruebas antes de integrar cambios a ramas principales.
- Separar las pruebas por tipo según su propósito: Unit Tests, Integration Tests, BDD Tests y System Tests.
- Mantener las pruebas unitarias enfocadas en componentes pequeños, rápidos y aislados.
- Usar pruebas de integración para validar la comunicación entre módulos, servicios y endpoints del backend.
- Utilizar Cucumber y archivos `.feature` para mantener la trazabilidad entre User Stories, Acceptance Criteria y pruebas automatizadas.
- Ejecutar el build del backend con Maven para verificar que el proyecto compile correctamente.
- Revisar los resultados de las pruebas antes de aprobar o integrar cambios.
- Utilizar Conventional Commits para diferenciar cambios de funcionalidad, corrección, configuración o pruebas, por ejemplo: `test: add bdd scenarios for inventory features`.
- Mantener las ramas organizadas bajo GitFlow para evitar cambios directos sobre ramas estables.
- Registrar evidencias de ejecución mediante capturas, resultados de pruebas y commits en GitHub.

7.1.2. Build & Test Suite Pipeline Components

Nuestro objetivo es Compilar el backend y validar automáticamente que los cambios superen todas las pruebas (Unit, Integration, BDD) antes de permitir un merge o despliegue.

Activadores (on): `push` y `pull_request` en ramas `feature/*`, `develop` y `testing`.

Pasos del pipeline:

1. `actions/checkout@v4` — Clonar el repositorio.
2. `actions/setup-java@v4` — Instalar JDK 17 y habilitar caché de Maven.
3. `mvn clean compile` — Validar sintaxis y dependencias sin ejecutar pruebas.
4. `mvn verify` — Ejecutar la suite completa de pruebas.
5. Subir artefactos de resultados (archivos `.xml`) para auditoría.

6. Reportar estado en el PR.

YAML de ejemplo ([.github/workflows/build-test-suite.yml](https://github.com/workflows/build-test-suite.yml)):

yaml name: Build & Test Suite

on: push: branches: [develop, testing, 'feature/**'] pull_request: branches: [develop, testing]

jobs: build-and-test: runs-on: ubuntu-latest

```
steps:
  - name: Checkout repository
    uses: actions/checkout@v4

  - name: Set up JDK 17
    uses: actions/setup-java@v4
    with:
      java-version: '17'
      distribution: 'temurin'
      cache: maven

  - name: Compile Backend
    run: mvn clean compile

  - name: Run Tests
    run: mvn verify

  - name: Upload test results
    if: always()
    uses: actions/upload-artifact@v4
    with:
      name: test-results
      path: |
        target/surefire-reports/*.xml
        target/failsafe-reports/*.xml
```

Puntos clave técnicos:

- Se fija el entorno en Java 17 tanto en el workflow como en el `<java.version>` del `pom.xml`, asegurando compatibilidad nativa con Spring Boot 3.x.
- Se utiliza `mvn verify` para ejecutar el ciclo completo. El `maven-surefire-plugin` se encarga de las pruebas unitarias y el `maven-failsafe-plugin` (previamente configurado en el POM) de las pruebas de integración y BDD.
- La caché de Maven reduce los tiempos de build. Los reportes de Surefire y Failsafe se exportan como artefactos para facilitar el análisis.
- Si la fase de compilación o alguna prueba falla, el workflow se detiene y las reglas del repositorio impiden el merge.

7.2. Continuous Delivery

7.2.1. Tools and Practices

El pipeline de entrega continua (CD) del sistema de gestión de inventario fue implementado utilizando GitHub Actions como herramienta principal de automatización y orquestación de despliegues. El proyecto backend desarrollado en Java utiliza Maven como gestor de dependencias y compilación.

Las prácticas aplicadas durante el proceso de integración y despliegue fueron las siguientes:

- **Automatización de despliegue:** cada cambio aprobado en la rama testing ejecuta automáticamente el pipeline de despliegue hacia el entorno de pruebas.
- **Validación previa:** el sistema únicamente permite el despliegue de versiones que hayan superado satisfactoriamente las pruebas unitarias implementadas con JUnit.
- **Control de versiones:** se utilizó GitHub para la gestión colaborativa del código fuente mediante ramas (main, testing y ramas feature).
- **Trazabilidad:** los resultados de compilación y pruebas son almacenados automáticamente como artefactos del pipeline para fines de monitoreo y auditoría.
- **Buenas prácticas DevOps:** se aplicó integración continua y despliegue continuo para asegurar estabilidad, control y rapidez en las actualizaciones del sistema.

7.2.2. Stages Deployment Pipeline Components

7.3. Continuous deployment

7.3.1. Tools and Practices

El proceso de Continuous Deployment (CD) automático hacia producción únicamente deberá habilitarse cuando la suite de pruebas y los *smoke scenarios* presenten un nivel adecuado de robustez, confiabilidad y cobertura.

Recomendaciones

- No habilitar despliegues automáticos hacia producción sin mecanismos de control adicionales, tales como:
 - aprobación humana (*approval gate*),
 - estrategias de *feature flags*,
 - validaciones previas obligatorias.
- En caso de habilitar CD automático:
 - la rama **main** deberá ejecutar un workflow automatizado responsable de:
 1. construir la aplicación,
 2. ejecutar pruebas Unit, Integration y Specs,
 3. generar y publicar artefactos,
 4. y realizar el despliegue automático hacia la plataforma objetivo (por ejemplo, Railway o Azure).
- El pipeline de despliegue deberá incluir mecanismos de rollback automático:
 - ante la detección de fallos en los *smoke tests* posteriores al despliegue,
 - el sistema deberá revertir automáticamente a la última versión estable disponible.

Consideraciones adicionales

- Todo despliegue a producción deberá generar trazabilidad mediante logs, métricas y registros de auditoría.
- Se recomienda integrar monitoreo continuo y alertas automáticas para detectar degradaciones funcionales o de rendimiento posteriores al despliegue.

7.3.2. Production Deployment Pipeline Components

- **Source Control Management:** Git
- **Build and compilation:** Vite, SWC (Speedy Web Compiler) y npm (Node Package Manager)
- **Artifact repository:** npm Public Registry (Registro público de npm)

7.4. Continuous Monitoring

7.4.1. Tools and Practices

7.4.2. Monitoring Pipeline Components

7.4.3. Alerting Pipeline Components

7.4.4. Notification Pipeline Components

Part III: Experiment-Driven Lifecycle

Capítulo VIII: Experiment-Driven Development

8.1. Experiment Planning

8.1.1. As-Is Summary

8.1.2. Raw Material: Assumptions, Knowledge Gaps, Ideas, Claims

8.1.3. Experiment-Ready Questions

8.1.4. Question Backlog

8.1.5. Experiment Cards

8.2. Experiment Design

8.2.1. Hypotheses

8.2.2. Domain Business Metrics

8.2.3. Measures

8.2.4. Conditions

8.2.5. Scale Calculations and Decisions

8.2.6. Methods Selection

8.2.7. Data Analytics: Goals, KPIs and Metrics Selection

8.2.8. Web and Mobile Tracking Plan

8.3. Experimentation

8.3.1. To-Be User Stories

8.3.2. To-Be Product Backlog

8.3.3. Pipeline-supported, Experiment-Driven To-Be Software Platform Lifecycle

8.3.3.1. To-Be Sprint Backlogs

8.3.3.2. Implemented To-Be Landing Page Evidence

8.3.3.3. Implemented To-Be Frontend-Web Application Evidence

8.3.3.4. Implemented To-Be Native-Mobile Application Evidence

8.3.3.5. Implemented To-Be RESTful API and/or Serverless Backend Evidence

8.3.3.6. Team Collaboration Insights

8.3.4. To-Be Validation Interviews

8.3.4.1. Diseno de Entrevistas

8.3.4.2. Registro de Entrevistas

8.4. Experiment Aftermath & Analysis

8.4.1. Analysis and Interpretation of Results

8.4.2. Re-scored and Re-prioritized Question Backlog

8.5. Continuous Learning

8.5.1. Shareback Session Artifacts: Learning Workflow

8.6. To-Be Software Platform Pre-launch

8.6.1. About-the-Product Intro Video

Conclusiones

Conclusiones y recomendaciones

Video App Validation

Video About-the-Team

Bibliografia

1. Dux Software. (2025, 9 abril). Dux Software: El Sistema de Gestión para tu Negocio.
<https://www.duxsoftware.com.ar/>
2. Mecalux. (s. f.). Store fulfillment. Mecalux.pe. <https://www.mecalux.pe/software/store-fulfillment>
3. SoftDolt. (s. f.). ▷Vendus: agiliza la gestión de tu punto de venta.
<https://www.softwaredoit.es/vendus/vendus.html>

Anexos

Enlace del repositorio repositorio: <https://github.com/upc-1ASI0732-2610-16879-Stoq>

Enlace directo del reporte: <https://github.com/upc-1ASI0732-2610-16879-Stoq/Report.git>